Replay Court — Repository Scaffold

Generated from Spec v0.4. Freeze confirmed. Build nothing outside this structure until the first green checkmark passes.

Directory Tree

```
replay-court/
│
├── spec/
│   └── v0.4/
│       └── replay-court-spec-v0.4.md        ← frozen, read-only
│
├── vectors/
│   ├── happy-path.json                       ← §15.1 inputs + expected outputs
│   ├── f01-invalid-engine.json
│   ├── f02-expired-proposal.json
│   ├── f03-duplicate-depth.json
│   ├── f04-skip-depth.json
│   ├── f05-policy-denied.json
│   ├── f06-bad-signature.json
│   ├── f07-bad-post-state.json
│   ├── f08-bad-pre-state.json
│   ├── f09-unknown-mutation-type.json
│   ├── f10-bad-semantic-version.json
│   ├── f11-bad-codec-version.json
│   ├── f12-tampered-payload.json
│   ├── f13-bad-context-id.json
│   └── f14-tombstoned-agent.json
│
├── typescript/
│   ├── src/
│   │   ├── types.ts                          ← all v0.4 structs
│   │   ├── hash.ts                           ← all §10 hash functions
│   │   ├── identity.ts                       ← ecrecover + identity binding law
│   │   ├── verify.ts                         ← §12 verification algorithm
│   │   └── index.ts
│   └── tests/
│       ├── hash.test.ts
│       ├── identity.test.ts
```

```
│       ├── happy-path.test.ts
│       └── failure-vectors.test.ts
│
├── rust/
│   ├── src/
│   │   ├── types.rs
│   │   ├── hash.rs
│   │   ├── identity.rs
│   │   ├── verify.rs
│   │   └── lib.rs
│   └── tests/
│       ├── hash_tests.rs
│       ├── identity_tests.rs
│       ├── happy_path.rs
│       └── failure_vectors.rs
│
├── python/
│   ├── replay_court/
│   │   ├── types.py
│   │   ├── hash.py
│   │   ├── identity.py
│   │   └── verify.py
│   └── tests/
│       ├── test_hash.py
│       ├── test_identity.py
│       ├── test_happy_path.py
│       └── test_failure_vectors.py
│
├── solidity/
│   ├── src/
│   │   ├── ReplayCourt.sol                  ← on-chain verification
│   │   ├── StateTransitionV1.sol            ← struct + hash
│   │   ├── MutationProposalV1.sol
│   │   ├── AuthorityContextV1.sol
│   │   ├── VerdictV1.sol
│   │   └── CourtOutcome.sol                 ← enum
│   └── test/
│       ├── HappyPath.t.sol
│       └── FailureVectors.t.sol
│
├── wasm-engine/
│   ├── src/
│   │   └── lib.rs                           ← reference mutation engine
│   ├── tests/
│   │   └── apply_mutation_test.rs
│   └── README.md                            ← build gate: Steps 1–5 must pass f
│
```

```
├── fuzz/
│   ├── fuzz_hash.rs                          ← differential: rust vs typescript
│   ├── fuzz_verify.rs                        ← arbitrary transition inputs
│   └── fuzz_authority_chain.rs               ← depth/role edge cases
│
├── property-tests/
│   ├── props_hash_determinism.rs             ← same input → same hash, always
│   ├── props_identity_binding.rs             ← recovered address == id, always
│   ├── props_transition_roundtrip.rs         ← encode → hash → verify → Transiti
│   └── props_invariant_completeness.rs       ← any invariant failure → non-Verif
│
└── parity/
    ├── run_all.sh                            ← executes happy-path vector across
    ├── compare.py                            ← diffs outputs; fails if any diver
    └── README.md
```

Vector Format

Each file in `/vectors/` follows this schema:

```
{
  "vector_id": "happy-path",
  "spec_version": "0.4",
  "description": "§15.1 normative happy path",
  "inputs": {
    "transition": { ... },
    "pre_state": { ... },
    "proposal": { ... },
    "payload_bytes_hex": "...",
    "authority_context": { ... },
    "mutation_engine_hash_hex": "..."
  },
  "expected": {
    "court_outcome": 0,
    "court_outcome_name": "TransitionVerified",
    "post_state": { ... },
    "authority_review": { ... },
    "intermediate_hashes": {
      "pre_state_hash": "...",
      "proposal_id": "...",
      "context_id": "...",
      "payload_hash": "...",
      "verdict_hash": "...",
      "transition_id": "..."
```

```
    }
  }
}
```

`intermediate_hashes` is mandatory. Every implementation must assert each one independently, not just the final `court_outcome`. This is what makes the vector useful for debugging cross-language divergence.

Implementation Gates

Gate 0 — Freeze

- `spec/v0.4/replay-court-spec-v0.4.md` committed, no further edits
- `vectors/` directory created with all 15 stub files

Gate 1 — State Hash Parity

All four implementations must produce identical output for:

```
state_hash(AgentGenomeStateV2 { ...§15.1 pre_state fields... })
```

No proceeding until this is green in Rust, TypeScript, Python, and Solidity.

This is the first green checkmark. It is the birth moment of the system.

Gate 2 — All Hash Functions

Every function in §10 produces identical output across all four implementations for identical inputs. Includes empty-list rule verification:

```
hash_authority_chain([]) == hash_attestations([]) == keccak256(abi.encode(uint256
```

Gate 3 — Identity Binding

`ecrecover(digest, sig)` → `bytes32(uint256(uint160(address)))` produces identical binding across all four implementations. Test with at least three known key pairs.

Gate 4 — Happy Path End-to-End

All intermediate hashes in `vectors/happy-path.json` are filled with concrete values (no placeholders). All four implementations assert each hash independently. `court_outcome ==`

`0` across all four.

Gate 5 — All 14 Failure Vectors

Every vector in `vectors/f*.json` is populated with concrete inputs. All four implementations produce exactly the expected `CourtOutcome` code for each. No partial passes.

Gate 6 — WASM Engine (unlocks after Gate 5)

Reference `mutation_engine_v1_test.wasm` built from `wasm-engine/src/lib.rs`. Loaded by Rust and TypeScript runtimes. Happy-path vector re-run with live WASM execution produces same `post_state_hash` as stored in `vectors/happy-path.json`.

Gate 7 — Parity Harness

`parity/run_all.sh` executes all 15 vectors across all four implementations and exits non-zero on any divergence. This is the CI gate. Nothing merges without it green.

Consensus-Critical Functions

These are equivalent to blockchain consensus code. They require:

- Deterministic output on all supported platforms (x86-64, arm64, wasm32)
- Property tests asserting determinism (same input → same output, always)
- Differential fuzzing across implementations
- No floating point, no randomness, no environment reads
- Eventually: formal invariants or formal verification

```
state_hash()
hash_transition_fields()
hash_proposal_fields()
hash_context_fields()
hash_verdict()
hash_authority_chain()
hash_attestations()
ecrecover_identity_binding()
```

Do not add feature flags, logging side-effects, or platform-specific branches to any of these functions. They are sacred.

What Is Not In Scope for v0.4 Implementation

These remain open but do not block Gate 1 through Gate 7:

- Mutation Semantic Registry (opcode standards, payload schemas)
- Reputation decay scoring formula
- Bond sizing for challenge protocol
- Policy language specification
- Multi-engine governance (how `mutation_engine_hash` is authorized)

These belong in v0.5 and beyond. Do not design them now. Implement v0.4 first.